

Payments & Money Logic

How CERVICED gets paid, how provider funds are protected, and how every money outcome is decided.

Reference document · 3 October 2026 · reflects decisions and work from this session. Not legal advice; sections marked with a legal flag need review before launch.

Status in one line. The machinery that *moves* money (checkout, capture, payouts, idempotency) is built and solid. The machinery that *returns* money (refunds) and the rules for no-shows are the work in progress; this document is the single description of both.

1. How CERVICED gets paid

CERVICED runs on a **client-side service fee**. The provider is paid for their service (or deposit); CERVICED adds its fee **on top** of that, charged to the client at checkout. The provider never pays the fee out of their own takings, and the provider never sees it.

The three numbers

Who	Pays / receives
Client	Service price (or deposit) plus the CERVICED service fee = their total
Provider	The service price (or deposit) only — their full entitlement
CERVICED	The service fee — its revenue, taken from the client, never from the provider

The fee itself

- Added on top of the client's subtotal, tiered by the full service value: under £100 = £1.99; £100 to £300 = £3.99; above £300 = £5.99.
- A deposit-only checkout uses the same tiers, based on the full service price — there is no separate flat deposit fee.
- In a mixed basket the fee is spread **across the pay-in-full bookings**, pro-rata.
- Every amount and the fee are computed **server-side** from the checkout batch. The app never sends an amount, a provider id or a Stripe account id that the server trusts.

How provider funds are protected: a provider's money is held by CERVICED after payment and only released to them 24 hours after the appointment ends (see the lifecycle). That hold window is also what makes refunds clean — the money hasn't left the platform yet.

2. The end-to-end money lifecycle

Every booking payment travels this path. Each step names the function or job that owns it.

#	Step	Owner	What happens
1	Price & reserve	prepare_checkout	Validates the basket, holds each slot (status on_hold, 10-min hold), writes the batch.
2	Add fee	apply_checkout_platform_fee	Adds the service fee on top, spread across pay-in-full bookings.
3	Authorise	create-payment-intent	Re-reads the amount server-side; blocks providers who aren't payout-ready; manual-capture intent.
4	Collect	Stripe PaymentSheet	The client authorises payment in the app.
5	Capture & confirm	finalize-payment-intent	Captures the money, then promotes bookings (on_hold -> pending or confirmed).
6	Hold payout	create_held_payout_on_finalize	Creates a held payout row: provider's share = client total minus fee. Releases 24h after the appointment ends.
7	Release	process_due_payouts -> release-payouts	Every 15 min: re-checks the booking, transfers the provider's share to their Stripe account.
8	Bank payout	Stripe	Stripe pays the provider's bank on its own schedule.

The fee (step 2) and the client's fee-inclusive total live in the booking and payout rows for settlement and refunding, but are not shown to the provider (see section 8).

3. Accounting invariants

These must always hold. Those enforced are guaranteed by a database constraint or a server check; those marked open are the work still outstanding.

Invariant	State
Provider payout = client total - service fee (per booking)	Enforced (DB CHECK)
Exactly one payout per eligible booking, safe on retries	Enforced (unique key + idempotency)
No underpayment — captured = amount owed; refund = amount paid	Enforced (server checks throw)
Refunds and transfers reconcile in either order	Enforced (reconcile logic)
Never pay out a cancelled / refunded / disputed booking	Enforced (re-checked at release)
Every captured booking has a ledger entry	Open — a provider off-boarding between authorise and finalize leaves no payout row

4. Provider payments experience

The provider's Payments & payouts area has four sections — and nothing else (no Invoices or Billing).

- **Overview** — available Stripe balance, funds clearing in Stripe, the next real bank payout (never invented when Stripe hasn't scheduled one), account actions, recent activity.
- **Booking payments** — history with each booking's **own share**, release status, and an authorised refund action. The client total and the CERVICED fee are not shown.
- **Payouts** — real Stripe bank payouts, separate from transfers into Stripe; amount, status, expected arrival, failure guidance. No in-app withdraw button, so unreleased funds can never be pulled.
- **Payment settings** — Stripe/verification status, bank & schedule, accepted-payment preferences, deposit settings, refund-policy link. Saving preserves unrelated policy fields.

What the provider must never see (product rule): CERVICED's service fee, the client's fee-inclusive total, any "client paid" figure that includes the fee, or CERVICED revenue. The provider sees only their own entitlement, refunds, and release status.

5. Client receipts & payment clarity

- The client sees the service/deposit amount, the CERVICED service fee, the total charged, any remaining balance due at the appointment, payment status and reference.
- A deposit is never shown as full payment; "paid" is keyed off real payment status.
- Success is shown only **after** the backend confirms; an ambiguous payment routes to a "check your bookings before paying again" state, never a duplicate charge.
- **Open item:** when a refund is issued, the client's receipt does not yet show "Refunded" — this is the next build item.

6. Refunds — what is automatic

A refund returns the client's full payment (fee included) from the platform balance; if the provider was already paid, their transfer is reversed so the platform nets out. The refund function is idempotent and a no-op on an unpaid booking.

Situation	Client's money	Status
Provider declines a paid booking	Full auto-refund	Built (migration, pending apply)
Provider ignores it until it expires	Full auto-refund	Built (migration, pending apply)
Client cancels their own booking	No refund (by policy)	Unchanged, intentional
Booking completes normally	No refund; provider paid	Normal path
No-show (either side)	See section 7	Design agreed; not yet built

How it's built: the refund fires at the exact moment of the provider-caused action (decline, or expiry of an ignored booking), where the cause is unambiguous — not by a later scan, because a provider-cancel and a client-cancel are indistinguishable after the fact.

Legal / policy flag. Automatic refunds on provider-caused non-completion, and the "client cancel = no refund" rule, are refund-policy decisions that should be reflected in the Terms.

7. No-show resolution — the no-referee model

No-show is handled so that **no case ever reaches the CERVICED owner**. Every outcome is decided by a timer, by a rule the client agreed to at booking, or by the client's own bank (chargeback). This replaces any "staff reviews it" step.

Why no proof is collected

The app cannot witness a physical absence, so it does not try to (no location or photo checks). Two facts make that unnecessary: the provider's payout is **held** through a dispute window, so money hasn't moved; and a provider who did the job **marks it complete** to get paid, so silence is the signal.

Client doesn't show

- After the grace period the provider marks the no-show; the agreed policy fee is charged automatically (deposit kept or stored card charged).
- If the client disagrees, their recourse is a chargeback with their bank — not the owner.

Provider doesn't show

- The client reports it; this **freezes auto-complete and the payout** so nothing reaches the provider.
- The provider gets a window to mark the booking complete.
- **Provider stays silent** -> the timer auto-refunds the client and cancels the held payout.
- **Provider marks it complete but the client disputes** -> **decided rule (Lean B): completion stands, no auto-refund, the client's recourse is their bank.**

What replaces the referee

- **Reputation, automatic:** strikes accrue on both sides; past a threshold an account is auto-suspended. No single case is reviewed.
- **Chargebacks:** the card network settles genuine money disputes off the owner's desk.

The one build item that makes this work: today the system auto-completes a booking after the appointment time, which would pay a provider who ghosted. A client no-show report must override auto-complete and freeze the payout until the provider responds.

Legal / policy flag. "Your recourse is your bank" for a client who paid and received nothing is the sharp edge: UK consumer law protects an undelivered service and cannot be fully contracted away. The Lean B default, the strike thresholds, and this wording need legal review and Terms coverage before launch.

8. No-show & cancellation policy options

A provider's policy is two decisions: how much to charge, and how that varies by notice. Recommendation: keep it to a notice window plus a late-cancel fee and a no-show fee — not a dozen knobs.

Fee amount options

Option	Meaning
Nothing	Free cancellation, no penalty
Keep the deposit	The deposit is the fee; nothing more charged
Percentage	Any % of the service price (50%, 100%, etc.)
Full amount	The whole service price

Flat fee	A fixed amount regardless of service price
----------	--

Tiered by notice (typical real policy)

Situation	Typical fee
Cancel early (before the notice window)	Free
Cancel late (inside the window)	Partial — often 50% or keep deposit
No-show (didn't attend)	Harshest — often 100%

Legal / policy flag. Charging 100% for a no-show is common, but under UK consumer law a fee must look like a genuine estimate of loss, not a penalty — the policy wording and Terms must frame it that way.

9. Security & the data boundary

- **Provider fee visibility (UI):** fixed — the provider payment breakdown no longer shows the client total or the platform fee, only the provider's share.
- **Provider fee visibility (data): still open** — a provider can read the fee and client total directly from the database. Closing it needs a column-level revoke plus a provider-safe view; the fee columns must stay for settlement, so they are hidden, not removed.
- **Owner finance:** there is no admin/owner role yet, so the owner finance dashboard is deferred; it must be built on a non-self-writable allowlist, never on the editable user role.
- **Live code safety:** the production Stripe edge functions existed only on disk, untracked; they are now committed so they can't be lost.
- **Hardening backlog:** payout/refund trigger functions are callable by any signed-in user (revoke), money tables keep default write grants (revoke), and six live migrations have no source file (write back from the live schema).

10. Decisions locked in

Decision	Call
Provider declines a paid booking	Auto-refund the client in full
Provider ignores until expiry	Auto-refund the client in full
Client cancels their own booking	No refund
Provider fee shown to providers	No — hide it; providers see only their share
Owner finance dashboard	Deferred
No-show handling	No-referee model; everything auto
No-show contested tie-break	Lean B — completion stands, client goes to their bank
No-show refund proof (location/photos)	Dropped — not used

11. Known gaps & launch blockers

Blocking before real money:

- Apply the auto-refund migration and verify it in Stripe test mode.
- Confirm the production flag that switches on tokenised Stripe payments; if it's off, the old raw-card mock form is live for real users — not PCI-safe.
- Close the provider fee data-boundary (column-level revoke + provider-safe view).
- Build the no-show pieces: client report freezes auto-complete & payout; apply the policy fee; Lean B default.

Important, not strictly blocking:

- Reschedule to a later date does not move the payout release date — a provider can be paid before the rescheduled appointment.
- Failed refunds and failed payouts are not retried or alerted; disputes after transfer are logged but not reversed. Both need the deferred ops/owner visibility.
- Write the six live-only payout migrations back into source; revoke public execute and default write grants on the money objects.
- Surface refund status on the client's receipt.
- Decide capture-upfront (today) vs capture-later (Fresha-style) — shapes how no-show fees are collected.

End of document. Generated for CERVICED · payments & money logic reference · treat legal-flagged items as to-review, not resolved.